

ESCUELA DE EDUCACIÓN SECUNDARIA TÉCNICA N.º 2 T2

Vicente López — Provincia de Buenos Aires

Laboratorio de Programación — 6.º Año, 2.ª División

INFORME TÉCNICO

Primer Avance del Proyecto

FinControl

INTEGRANTES DEL EQUIPO

Terry Calero

Franco Cannizaro

Leonidas Druetta

1. Resumen:

El presente informe técnico documenta el desarrollo del primer avance del proyecto FinControl, una aplicación web de administración financiera personal elaborada como trabajo integrador de la asignatura Laboratorio de Programación, correspondiente al 6.º año de la carrera de Técnico en Informática de la EEST N.º 2 T2 de Vicente López.

FinControl es una aplicación desarrollada íntegramente con tecnologías web estándar: HTML5 para la estructuración del contenido, CSS3 para el diseño visual y el comportamiento responsivo, y JavaScript para la implementación de la lógica de negocio y los cálculos automáticos. La aplicación no depende de librerías externas ni de frameworks adicionales, lo que pone en evidencia el dominio de las tecnologías base por parte del equipo desarrollador.

El sistema permite al usuario registrar su sueldo mensual, ingresar gastos individuales con su correspondiente fecha, descripción y categoría, visualizar el historial de movimientos en una tabla interactiva, consultar un resumen financiero en tiempo real mediante tarjetas de información, obtener un diagnóstico automático de su situación financiera según el porcentaje del sueldo consumido, y acceder a los promedios de gasto diario, semanal y mensual.

Este documento desarrolla en profundidad todos los aspectos técnicos del sistema: análisis del problema, objetivos, alcance, requerimientos, tecnologías, arquitectura, funcionamiento, interfaz, lógica de programación, estructura del proyecto, buenas prácticas aplicadas y posibles líneas de mejora futura.

Palabras clave: administración financiera personal, aplicación web, HTML5, CSS3, JavaScript, programación orientada a eventos, MVP, EEST N.º 2.

2. Introduccion

La administración responsable de las finanzas personales constituye una habilidad fundamental en la vida cotidiana de cualquier individuo. Sin embargo, estudios de educación financiera señalan que únicamente tres de cada diez personas organizan activamente sus ingresos y gastos. Esta realidad evidencia una brecha significativa entre la necesidad de planificar el presupuesto personal y las herramientas disponibles o conocidas por el ciudadano común.

En este contexto, el equipo conformado por Terry Calero, Franco Cannizaro y Leonidas Druetta, estudiantes del 6.º año de la especialidad Técnico en Informática de la EEST N.º 2 T2 de Vicente López, desarrolló FinControl: una aplicación web diseñada para brindar una solución sencilla, accesible y eficiente al problema del descontrol financiero personal.

Este proyecto surge como respuesta concreta a la consigna del primer avance de la asignatura Laboratorio de Programación, que requiere la presentación de un Producto Mínimo Viable (MVP) que demuestre la integración entre la interfaz de usuario y la lógica del sistema, respetando buenas prácticas de programación.

FinControl se construyó sin depender de herramientas externas, bases de datos o servidores: toda la lógica opera del lado del cliente (client-side), lo que permite su ejecución en cualquier navegador web moderno sin necesidad de instalación ni conexión a internet. Esta decisión de diseño responde tanto a las limitaciones propias del contexto educativo como a la intención de demostrar el dominio de las tecnologías fundamentales de la web.

El presente informe describe en detalle cada componente del sistema, desde la justificación del problema hasta las posibles líneas de evolución futura, permitiendo evaluar tanto la calidad técnica del desarrollo como el nivel de comprensión conceptual alcanzado por el equipo.

3. Análisis y Justificación del Problema

La problemática central que motivó el desarrollo de FinControl puede ser sintetizada en cuatro ejes principales identificados durante la etapa de análisis:

- Falta de organización financiera: gran parte de la población no lleva un registro formal de sus ingresos y egresos mensuales. Esta ausencia de organización impide la planificación a corto y mediano plazo.
- Desconocimiento de los gastos diarios: sin un sistema de registro, el usuario pierde conciencia del costo acumulado de pequeños gastos cotidianos, que en conjunto representan una porción significativa del presupuesto mensual.
- Dificultad para controlar el presupuesto mensual: sin un punto de referencia claro (el sueldo disponible) y sin un seguimiento de los egresos, resulta muy difícil evaluar si el nivel de gasto es sostenible o no.
- Escasa planificación económica: la ausencia de datos concretos sobre el propio comportamiento financiero impide proyectar objetivos de ahorro, identificar gastos prescindibles o anticipar situaciones de déficit.

Frente a estas problemáticas, las soluciones tecnológicas existentes suelen presentar obstáculos de adopción: aplicaciones complejas con demasiadas funcionalidades, herramientas que requieren registro o suscripción, o aplicaciones que no están adaptadas al contexto económico local. FinControl propone un abordaje diferente: una herramienta minimalista, que funciona en cualquier navegador web sin instalación, con un flujo de uso intuitivo que no exige conocimientos previos de finanzas.

La justificación del desarrollo de esta aplicación radica, por lo tanto, en la convergencia de dos necesidades concretas: por un lado, la necesidad social de contar con herramientas simples para la gestión del dinero personal; por otro, la necesidad académica de integrar los conocimientos adquiridos en programación web aplicándolos a un problema real y significativo.

4. Objetivo General

Desarrollar una aplicación web que permita al usuario registrar su sueldo mensual, ingresar y controlar sus gastos personales de manera organizada por categoría y fecha, y visualizar en tiempo real un resumen financiero automatizado que facilite la toma de decisiones económicas informadas.

Este objetivo general implica la integración de los conocimientos de programación web adquiridos durante la formación técnica, demostrando capacidad para articular la capa de presentación (interfaz de usuario), la lógica de negocio (cálculos y procesamiento de datos) y la experiencia de usuario (navegación, retroalimentación y accesibilidad) en un único producto funcional.

5. Objetivos Específicos

Los siguientes objetivos específicos desagregan el propósito general del proyecto en metas concretas y verificables:

- Implementar un formulario de registro del sueldo mensual con validación de datos, que sirva como parámetro base para todos los cálculos del sistema.
- Desarrollar un formulario de carga de gastos individuales que permita ingresar fecha, descripción, categoría y monto, con validaciones que garanticen la integridad de los datos.
- Clasificar cada gasto registrado dentro de un sistema de nueve categorías predefinidas: Alimentación, Transporte, Vivienda, Salud, Entretenimiento, Educación, Ropa, Servicios y Otros.
- Calcular y mostrar en tiempo real el dinero disponible como resultado de la diferencia entre el sueldo mensual y el total de gastos acumulados.
- Calcular y mostrar el porcentaje del sueldo mensual que ya ha sido consumido por los gastos registrados.
- Calcular y mostrar promedios de gasto diario, semanal (proyección a 7 días) y mensual (proyección a 30 días) basados en los datos reales ingresados por el usuario.
- Generar y mostrar automáticamente un mensaje de diagnóstico del estado financiero del usuario, diferenciando tres niveles: situación buena (menos del 50% del sueldo gastado), situación moderada (entre 50% y 80%), y situación crítica (más del 80%).
- Implementar un historial tabular de todos los gastos registrados, con posibilidad de eliminar registros individuales de forma dinámica.
- Garantizar que la interfaz se adapte correctamente a distintos tamaños de pantalla mediante diseño responsivo.

6. Alcance del Proyecto

6.1. Alcance incluido

El presente primer avance abarca la construcción completa del Producto Mínimo Viable (MVP) de FinControl. El sistema implementado en esta etapa comprende:

- Interfaz web desarrollada en un único archivo HTML con estilos CSS integrados y lógica JavaScript embebida.
- Módulo de configuración del sueldo mensual con validación de entrada.
- Módulo de registro de gastos con campos de fecha, descripción, categoría y monto, incluyendo validaciones de formulario.
- Módulo de historial de gastos con tabla dinámica y función de eliminación individual.
- Módulo de resumen financiero con cuatro tarjetas de información (sueldo, total gastado, dinero disponible y porcentaje utilizado).
- Módulo de promedios de gasto (diario, semanal y mensual).
- Sistema de diagnóstico automático del estado financiero con tres niveles de alerta.
- Diseño responsivo compatible con dispositivos móviles, tablets y computadoras de escritorio.

6.2. Alcance excluido

Las siguientes funcionalidades se encuentran fuera del alcance del primer avance, quedando previstas para etapas posteriores del desarrollo:

- Persistencia de datos: la aplicación no utiliza bases de datos ni almacenamiento local (localStorage), por lo que los datos se pierden al cerrar o recargar la página.
- Sistema de usuarios: no existe autenticación, registro ni inicio de sesión.
- Gráficos estadísticos de distribución de gastos por categoría.
- Exportación de datos en formato PDF o Excel.
- Sistema de metas de ahorro o recordatorios de pagos.
- Versión nativa para dispositivos móviles (aplicación Android o iOS).

7. Requerimientos Funcionales

Los requerimientos funcionales describen el comportamiento concreto que el sistema debe exhibir ante las acciones del usuario. A continuación se detallan todos los requerimientos funcionales identificados:

ID	Requerimiento Funcional	Descripción
RF-01	Registro de sueldo mensual	El sistema debe permitir al usuario ingresar un valor numérico positivo como sueldo mensual. El ingreso debe ser validado antes de ser aceptado.
RF-02	Actualización de tarjetas	Al registrar el sueldo, el sistema debe actualizar automáticamente las cuatro tarjetas del resumen financiero.
RF-03	Registro de gasto	El sistema debe permitir agregar un gasto con los campos: fecha (tipo date), descripción (texto, máx. 60 caracteres), categoría (selección de lista) y monto (numérico positivo).
RF-04	Validación de formulario	Todos los campos del formulario de gasto son obligatorios. El sistema debe mostrar un mensaje de alerta si algún campo no está completo o contiene un valor inválido.
RF-05	Categorización de gastos	El sistema debe clasificar cada gasto en una de nueve categorías: Alimentación, Transporte, Vivienda, Salud, Entretenimiento, Educación, Ropa, Servicios u Otros.
RF-06	Visualización del historial	El sistema debe mostrar todos los gastos registrados en una tabla con columnas de Fecha, Descripción, Categoría, Monto y acción de Eliminar.
RF-07	Eliminación de gasto	El sistema debe permitir eliminar un gasto individual del historial. La eliminación debe actualizar automáticamente todos los cálculos y la tabla.
RF-08	Cálculo del saldo disponible	El sistema debe calcular y mostrar la diferencia entre el sueldo mensual y la suma total de los gastos registrados.
RF-09	Cálculo del porcentaje utilizado	El sistema debe calcular y mostrar el porcentaje del sueldo mensual que ha sido consumido por los gastos registrados.
RF-10	Cálculo de promedios	El sistema debe calcular el promedio diario de gasto en función de los días únicos con al menos un registro; y proyectar ese promedio a 7 y 30 días para obtener el promedio semanal y mensual respectivamente.

RF-11	Diagnóstico automático	El sistema debe generar un mensaje de estado financiero automático según el porcentaje del sueldo consumido: bueno (<50%), moderado (50%-80%) o crítico (>80%).
RF-12	Navegación por secciones	El sistema debe ofrecer un menú de navegación con enlaces a las secciones Inicio, Presupuesto, Registrar Gasto e Historial.

8. Requerimientos No Funcionales

Los requerimientos no funcionales establecen las condiciones de calidad, rendimiento y restricciones técnicas que el sistema debe satisfacer más allá de sus funcionalidades específicas:

ID	Categoría	Requerimiento
RNF-01	Usabilidad	La interfaz debe ser intuitiva, sin curva de aprendizaje significativa. Un usuario sin conocimientos técnicos debe poder utilizar el sistema sin instrucciones previas.
RNF-02	Compatibilidad	La aplicación debe funcionar correctamente en los navegadores web modernos más utilizados: Google Chrome, Mozilla Firefox, Microsoft Edge y Safari.
RNF-03	Responsividad	El diseño debe adaptarse a distintos tamaños de pantalla mediante CSS Grid y Media Queries, garantizando una experiencia adecuada en dispositivos móviles, tablets y escritorio.
RNF-04	Rendimiento	Dado que toda la lógica opera del lado del cliente, las operaciones de cálculo y actualización de la interfaz deben ser prácticamente instantáneas.
RNF-05	Mantenibilidad	El código fuente debe estar organizado en secciones claramente delimitadas y comentadas, facilitando su comprensión, modificación y extensión futura.
RNF-06	Portabilidad	La aplicación debe poder ejecutarse en cualquier dispositivo con navegador web moderno, sin requerir instalación, conexión a internet ni servidor web.
RNF-07	Autonomía tecnológica	El sistema no debe depender de librerías, frameworks ni CDN externos. Todo el código (HTML, CSS y JavaScript) debe estar contenido en el archivo fuente.
RNF-08	Accesibilidad básica	Los formularios deben usar etiquetas <label> vinculadas correctamente a sus campos de entrada, mejorando la accesibilidad para lectores de pantalla.
RNF-09	Seguridad básica	El sistema debe validar los datos ingresados antes de procesarlos, evitando operaciones con valores vacíos, negativos o no numéricos.

9. Tecnologías Utilizadas

9.1. HTML5

HTML (HyperText Markup Language) en su versión 5 es el lenguaje estándar para la estructuración y presentación del contenido web. En FinControl, HTML5 cumple la función de definir la arquitectura semántica de la aplicación, organizando sus componentes en secciones identificadas mediante los atributos id y los elementos estructurales nativos (header, nav, main, section, footer).

Se utilizaron elementos HTML5 específicos para los formularios: el tipo date para el campo de fecha (que activa el selector nativo de fechas del navegador), el tipo number para los campos de sueldo y monto (que restringe el ingreso a valores numéricos), y el elemento select para la lista de categorías de gastos. La tabla de historial fue implementada mediante los elementos estándar table, thead, tbody, th y td.

9.2. CSS3

CSS (Cascading Style Sheets) en su versión 3 es el lenguaje de estilos que define la presentación visual de la aplicación. En FinControl, CSS3 cumple un rol central: define la paleta de colores del sistema (azul marino #2c3e50 y azul acento #2980b9), establece las tipografías, espaciados y bordes, e implementa el sistema de layout responsivo.

Para el layout del resumen financiero y los promedios se utilizó CSS Grid, que permite organizar las tarjetas en una cuadrícula de columnas iguales que se contrae a una sola columna en pantallas pequeñas mediante la directiva @media (max-width: 600px). Las transiciones CSS fueron aplicadas a los botones y los campos de formulario para mejorar la retroalimentación visual al usuario.

Los estilos de estado del mensaje financiero (msg-bajo, msg-moderado, msg-alto) utilizan colores semánticos (verde, amarillo y rojo) que refuerzan visualmente el diagnóstico del sistema, apelando a la convención de colores de alerta ampliamente reconocida por los usuarios.

9.3. JavaScript

JavaScript es el lenguaje de programación del lado del cliente que implementa toda la lógica de negocio de FinControl. El código JavaScript de la aplicación está estructurado en cinco funciones principales: guardarSueldo(), agregarGasto(), eliminarGasto(), actualizarTabla() y actualizarResumen(). Además, se declaran dos variables globales: sueldo (tipo Number) y gastos (tipo Array de objetos).

La programación utilizada es de tipo orientada a eventos: cada función es invocada por un evento del usuario (clic en un botón, carga de la página). Se emplean los métodos nativos del DOM para leer los valores de los campos de formulario (getElementById, value, valueAsDate), modificar el contenido de los elementos (textContent, innerHTML) y manipular las clases CSS (className, style.display) para mostrar u ocultar el mensaje de estado.

9.4. Visual Studio Code

Visual Studio Code fue el entorno de desarrollo integrado (IDE) utilizado por el equipo para la escritura, edición y depuración del código fuente. Su soporte nativo para HTML, CSS y JavaScript, combinado con extensiones de autocompletado y previsualización en tiempo real, facilitó el desarrollo iterativo del proyecto.

10. Arquitectura del Sistema

10.1. Modelo general

FinControl adopta una arquitectura monolítica de una sola página (SPA – Single Page Application en su forma más simple), donde todos los componentes del sistema (presentación, estilos y lógica) se concentran en un único archivo HTML. Esta decisión de diseño es coherente con el contexto y los objetivos del proyecto: eliminar dependencias externas, facilitar la portabilidad y concentrar la demostración de habilidades en el dominio de las tecnologías base.

La arquitectura puede describirse como de tres capas lógicas integradas en un único archivo:

- Capa de presentación (HTML): define la estructura semántica de la interfaz mediante secciones identificadas por atributos id. Cada sección corresponde a un módulo funcional del sistema.
- Capa de estilos (CSS): gestiona la presentación visual, el layout responsivo y los estados visuales de los componentes. Los estilos están escritos dentro del bloque `<style>` en el `<head>` del documento.
- Capa de lógica (JavaScript): implementa el comportamiento dinámico de la aplicación. El código está contenido en un bloque `<script>` al final del `<body>`, siguiendo la buena práctica de diferir la carga de scripts para no bloquear el renderizado del HTML.

10.2. Flujo de datos

El flujo de datos de la aplicación sigue el siguiente recorrido: el usuario ingresa datos a través de los formularios HTML; JavaScript lee esos datos mediante `getElementById()`, los valida, los procesa y los almacena en las variables globales `sueldo` y `gastos[]`; finalmente, las funciones `actualizarTabla()` y `actualizarResumen()` leen el estado actual de esas variables y actualizan el DOM para reflejar los cambios en la interfaz.

Este modelo de flujo unidireccional (entrada → procesamiento → salida) garantiza la coherencia de los datos mostrados en todo momento, ya que cualquier modificación al estado (agregar o eliminar un gasto, cambiar el sueldo) desencadena una actualización completa de todos los indicadores.

11. Descripción del Funcionamiento del Sistema

El sistema FinControl opera mediante un ciclo de cuatro etapas que el usuario ejecuta de forma natural e intuitiva:

Etapas 1: Configuración del presupuesto

El usuario accede a la sección «Presupuesto mensual» e ingresa su sueldo en el campo correspondiente. Al presionar el botón «Guardar sueldo», la función `guardarSueldo()` valida que el valor sea un número positivo y lo almacena en la variable global `sueldo`. Inmediatamente, se invoca `actualizarResumen()` para reflejar el nuevo sueldo en la tarjeta correspondiente y recalcular todos los indicadores.

Etapas 2: Registro de gastos

En la sección «Registrar gasto», el usuario completa el formulario indicando fecha (inicializada automáticamente con la fecha actual del sistema), descripción del gasto, categoría y monto. Al presionar «+ Agregar gasto», la función `agregarGasto()` valida todos los campos. Si la validación es exitosa, crea un objeto JavaScript con las propiedades del gasto (incluyendo un identificador único generado con `Date.now()`) y lo incorpora al array global `gastos[]`. A continuación, limpia los campos del formulario y actualiza la tabla y el resumen.

Etapa 3: Visualización y seguimiento

La tabla de historial muestra todos los gastos registrados en orden de ingreso. Cada fila presenta la fecha formateada en formato DD/MM/AAAA (adaptando el formato ISO AAAA-MM-DD devuelto por el campo `date`), la descripción, la categoría, el monto formateado con separadores de miles según el locale «es-AR», y un botón de eliminación individual.

Las cuatro tarjetas de la sección de presupuesto se actualizan en tiempo real: Sueldo mensual, Total gastado, Dinero disponible y Porcentaje utilizado. Debajo de las tarjetas, el mensaje de diagnóstico automático se muestra y actualiza según los umbrales definidos.

Etapa 4: Análisis de promedios

La sección «Promedios de gasto» calcula el gasto promedio diario utilizando como divisor la cantidad de días únicos con al menos un registro (evitando divisiones por cero con un mínimo de 1). El promedio semanal se proyecta multiplicando el diario por 7, y el mensual por 30. Estos indicadores permiten al usuario anticipar su nivel de gasto futuro.

12. Interfaz de Usuario

12.1. Diseño visual

La interfaz de FinControl adopta un diseño limpio y minimalista, con una paleta de colores institucional basada en tonos de azul marino (#2c3e50 para el encabezado y pie de página, #1a252f para la barra de navegación, #2980b9 como color de acento para botones y títulos de sección). El fondo de la página utiliza un gris muy claro (#f0f2f5) que contrasta con las secciones blancas, generando una jerarquía visual clara.

Las tarjetas de resumen emplean colores semánticos: azul para el sueldo mensual, rojo para el total gastado, verde para el dinero disponible y naranja para el porcentaje utilizado. Esta codificación cromática permite al usuario interpretar el estado financiero de un vistazo sin necesidad de leer los valores numéricos.

12.2. Estructura de navegación

La barra de navegación fija en la parte superior contiene cuatro enlaces de ancla que permiten desplazarse directamente a cada sección de la aplicación: Inicio, Presupuesto, Registrar Gasto e Historial. Los enlaces poseen un efecto hover con cambio de color de fondo, proporcionando retroalimentación visual al usuario.

12.3. Formularios

Los formularios de la aplicación utilizan el sistema de etiquetas label vinculadas a sus campos mediante el atributo for. Los campos de texto y número comparten un estilo visual unificado, con bordes grises que se resaltan en azul al recibir el foco (pseudoclase :focus). El campo de fecha se inicializa automáticamente con la fecha actual del sistema, reduciendo la fricción de uso.

12.4. Diseño responsivo

El diseño responsivo se implementa mediante dos mecanismos complementarios: CSS Grid para los layouts de tarjetas, formularios y promedios (con columnas que se adaptan automáticamente al ancho

disponible), y una Media Query con punto de quiebre en 600px de ancho que convierte todos los layouts de dos o tres columnas en una sola columna para pantallas pequeñas.

13. Lógica de Programación Implementada

13.1. Variables globales y estructura de datos

El sistema mantiene dos variables globales en tiempo de ejecución. La variable sueldo almacena un valor de tipo Number que representa el ingreso mensual del usuario. La variable gastos es un Array de objetos JavaScript, donde cada objeto posee las propiedades: id (Number, generado con Date.now()), fecha (String en formato AAAA-MM-DD), descripcion (String), categoria (String) y monto (Number).

13.2. Función guardarSueldo()

Esta función lee el valor del campo de entrada con id «sueldo» mediante parseFloat(), garantizando que el resultado sea un número de punto flotante. Verifica que el valor no sea NaN y sea mayor que cero antes de asignarlo a la variable global. En caso de invalidez, muestra un alert() con el mensaje de error correspondiente.

13.3. Función agregarGasto()

Implementa la lógica de validación y registro de nuevos gastos. Lee los cuatro campos del formulario, aplica validaciones individuales con mensajes de error específicos para cada campo, y si todas las validaciones son superadas, crea un objeto gasto con un identificador único basado en Date.now() y lo incorpora al array gastos[] mediante el método push(). Finaliza limpiando los campos del formulario y reinvocando las funciones de actualización.

13.4. Función eliminarGasto(id)

Recibe el identificador único del gasto a eliminar y aplica el método filter() sobre el array gastos[] para generar un nuevo array que excluye el elemento con el id correspondiente. Esta implementación es funcional e inmutable: no

modifica el array original in-place, sino que reasigna la variable global a un nuevo array filtrado. Finaliza actualizando la tabla y el resumen.

13.5. Función actualizarTabla()

Gestiona el renderizado dinámico de la tabla de historial. Evalúa si el array `gastos[]` está vacío: si es así, inserta la fila de estado vacío («Todavía no hay gastos registrados»). En caso contrario, recorre el array con un bucle `for` y construye el HTML de cada fila de forma programática. La fecha se reformatea de AAAA-MM-DD a DD/MM/AAAA mediante el método `split()` y reordenamiento de los segmentos del string. Los montos se formatean con `toLocaleString('es-AR')` para aplicar los separadores de miles del español argentino.

13.6. Función actualizarResumen()

Centraliza todos los cálculos del sistema: suma los montos del array `gastos[]` mediante un bucle acumulativo, calcula el dinero disponible como $(\text{sueldo} - \text{totalGastado})$, calcula el porcentaje como $(\text{totalGastado} / \text{sueldo}) * 100$ con protección contra división por cero, calcula los días únicos con gasto mediante el constructor `Set()` aplicado sobre el array de fechas, y proyecta los promedios. Actualiza las cuatro tarjetas, los tres promedios y el mensaje de diagnóstico mediante accesos directos al DOM con `getElementById()` y la propiedad `textContent`.

13.7. Sistema de diagnóstico automático

El diagnóstico se implementa mediante una estructura de decisión `if / else if / else` con los siguientes umbrales: porcentaje menor a 50 activa la clase CSS `msg-bajo` con ícono  y mensaje positivo; porcentaje entre 50 y 80 activa `msg-moderado` con ícono  y mensaje de advertencia; porcentaje mayor o igual a 80 activa `msg-alto` con ícono  y mensaje de alerta crítica. Si no hay sueldo cargado, el div del mensaje permanece oculto mediante `style.display = 'none'`.

14. Estructura del Proyecto

La estructura de archivos del proyecto FinControl es la siguiente:

Archivo / Carpeta	Descripción
index.html	Archivo principal y único de la aplicación. Contiene la estructura HTML, los estilos CSS (bloque <code><style></code>) y la lógica JavaScript (bloque <code><script></code>).
FinControl.pptx	Presentación de diapositivas utilizada para la exposición oral del primer avance del proyecto.
README.md (recomendado)	Archivo de documentación breve del proyecto, con instrucciones de uso y descripción general del sistema.

La decisión de concentrar toda la aplicación en un único archivo HTML responde a la naturaleza del MVP y a los objetivos pedagógicos del primer avance: demostrar la integración de las tres tecnologías fundamentales de la web (HTML, CSS y JavaScript) en un producto funcional y coherente. En fases posteriores del proyecto, se recomienda separar los estilos en un archivo `styles.css` y la lógica en un archivo `app.js`, siguiendo el principio de separación de responsabilidades.

15. Buenas Prácticas de Programación Aplicadas

El desarrollo de FinControl incorporó de manera sistemática las siguientes buenas prácticas de programación, en consonancia con los estándares profesionales de la industria web y con los criterios evaluativos de la asignatura:

15.1. Código comentado y documentado

Todas las funciones JavaScript están precedidas por un bloque de comentario que indica su propósito. Los bloques de código CSS están delimitados por comentarios de sección (`/* ===== SECCIÓN ===== */`) que facilitan la navegación dentro del archivo. Esta práctica reduce significativamente el tiempo necesario para comprender y mantener el código.

15.2. Nomenclatura en español y consistente

Los identificadores de variables (`sueldo`, `gastos`, `totalGastado`, `promDiario`), funciones (`guardarSueldo`, `agregarGasto`, `eliminarGasto`, `actualizarTabla`, `actualizarResumen`) y clases CSS (`tarjeta`, `prom-caja`, `btn-azul`, `msg-moderado`) siguen una nomenclatura en español coherente con el idioma del proyecto, clara, descriptiva y libre de ambigüedades.

15.3. Separación de responsabilidades

Aunque todo el código reside en un único archivo, la separación lógica de responsabilidades está claramente implementada: el HTML define la estructura, el CSS define la presentación y el JavaScript define el comportamiento. Dentro del JavaScript, cada función tiene una única responsabilidad bien definida, evitando funciones que realicen múltiples tareas no relacionadas.

15.4. Validación de entradas

Todas las entradas del usuario son validadas antes de ser procesadas. Las funciones `guardarSueldo()` y `agregarGasto()` verifican la completitud y la corrección de los datos mediante `isNaN()`, comparaciones con 0 y verificación de strings vacíos. Este enfoque de validación defensiva previene el ingreso de datos inconsistentes que podrían producir resultados erróneos en los cálculos.

15.5. Uso de identificadores únicos

Cada gasto registrado recibe un identificador único generado mediante `Date.now()`, que devuelve el número de milisegundos transcurridos desde el 1 de enero de 1970. Este enfoque garantiza que no existan dos gastos con el mismo identificador en una sesión normal de uso, lo que hace que la función de eliminación por id sea segura y predecible.

15.6. Formato localizado de números

Los montos se muestran con separadores de miles y decimal correspondientes al español argentino mediante el método `toLocaleString('es-AR')`, mejorando significativamente la legibilidad de los valores monetarios y adaptando la aplicación al contexto cultural del usuario objetivo.

15.7. Inicialización automática de la fecha

Al cargar la página, el campo de fecha del formulario de registro de gastos se inicializa automáticamente con la fecha actual del sistema mediante `document.getElementById('fecha').valueAsDate = new Date()`. Esta inicialización reduce el esfuerzo requerido al usuario y evita registros con fechas vacías o incorrectas.

15.8. Código sin dependencias externas

La aplicación no importa ninguna librería, framework ni recurso externo. Todo el código es original y autocontenido. Esta decisión no solo garantiza la portabilidad y la ausencia de dependencias de conectividad, sino que también demuestra un dominio genuino de las tecnologías base por parte del equipo.

16. Posibles Mejoras Futuras

El equipo ha identificado un conjunto de funcionalidades y mejoras que podrían incorporarse en etapas posteriores del proyecto, potenciando significativamente la utilidad y la solidez técnica del sistema:

16.1. Persistencia de datos con localStorage

La principal limitación del MVP actual es la ausencia de persistencia: los datos se pierden al cerrar o recargar el navegador. La incorporación de la API `localStorage` de JavaScript permitiría guardar el sueldo y el array de gastos en el almacenamiento local del navegador, recuperándolos automáticamente en cada visita sin necesidad de un servidor ni una base de datos.

16.2. Base de datos y sistema de usuarios

Para una versión más avanzada del sistema, se podría incorporar un backend con una base de datos relacional (por ejemplo, MySQL o PostgreSQL) y un sistema de autenticación de usuarios (registro e inicio de sesión). Esto permitiría que múltiples usuarios accedan a sus datos personales desde cualquier dispositivo.

16.3. Gráficos estadísticos

La incorporación de gráficos de torta o de barras que representen la distribución del gasto por categoría añadiría una capa de análisis visual muy valiosa. Librerías como Chart.js o D3.js, integradas mediante un tag <script> de CDN, permitirían implementar esta funcionalidad con relativa sencillez.

16.4. Exportación de datos

La posibilidad de exportar el historial de gastos y el resumen financiero en formato PDF o en una hoja de cálculo Excel (formato .xlsx) aumentaría significativamente la utilidad del sistema para usuarios que deseen llevar un registro externo de sus finanzas.

16.5. Metas de ahorro y recordatorios

La incorporación de un módulo de metas de ahorro (donde el usuario defina un objetivo de ahorro mensual) y un sistema de recordatorios de pagos fijos (alquiler, servicios, cuotas) completaría el perfil de una herramienta de planificación financiera personal integral.

16.6. Aplicación móvil nativa

Una versión nativa para Android o iOS, desarrollada con tecnologías como React Native o Flutter, o bien una Progressive Web App (PWA) con soporte offline, permitiría a los usuarios acceder a FinControl directamente desde la pantalla de inicio de su teléfono, con la experiencia de una aplicación nativa.

17. Conclusiones

El desarrollo del primer avance de FinControl representó para el equipo un ejercicio integral de aplicación de los conocimientos adquiridos en la asignatura Laboratorio de Programación. La construcción del sistema implicó tomar decisiones de diseño técnico concretas, resolver problemas de programación reales y producir un artefacto de software funcional que responde a una necesidad identificada en el contexto cotidiano.

Desde el punto de vista técnico, el proyecto demostró que es posible construir una aplicación web completamente funcional utilizando únicamente HTML5, CSS3 y JavaScript, sin depender de ninguna tecnología adicional. Esta restricción autoimpuesta no solo fue satisfecha con éxito, sino que permitió evidenciar un dominio genuino de las tres tecnologías base de la web.

La aplicación logró cumplir con todos los objetivos planteados para esta etapa: registrar el sueldo mensual, gestionar gastos categorizados, calcular indicadores financieros en tiempo real, proyectar promedios de consumo y generar un diagnóstico automático del estado financiero del usuario. Asimismo, la interfaz resultante es visualmente coherente, responsiva y accesible.

Desde el punto de vista de las buenas prácticas de programación, el equipo incorporó comentarios descriptivos, nomenclatura consistente, validación de entradas, identificadores únicos, formateo localizado de valores y código libre de dependencias externas. El resultado es un MVP que, además de ser funcional, es mantenible y extensible.

El proyecto sienta las bases para una segunda etapa en la que se podrían incorporar la persistencia de datos mediante localStorage, gráficos estadísticos, un sistema de usuarios y exportación de informes. Cada uno de estos módulos puede incorporarse progresivamente sobre la arquitectura actual, dado que el sistema fue diseñado pensando en su escalabilidad.

En síntesis, FinControl es una solución simple, eficiente y accesible que demuestra que la tecnología, cuando es comprendida y aplicada con criterio, puede contribuir a resolver problemas concretos de la vida cotidiana. El

equipo considera que el objetivo del primer avance ha sido satisfecho en su totalidad y que el proyecto se encuentra en condiciones de avanzar hacia etapas de mayor complejidad.

